

EXPLAIN IT LIKE I'M FIVE

The Render Farm, Explained Simply

The same system as the design document, but told slowly and from the very beginning, assuming you know nothing about computers, servers, or queues. Every word is explained the first time it is used.

The promise of this document: if you can picture a busy restaurant kitchen, you can understand this entire system. We will build the whole picture out of that one idea, and only afterward attach the real computer words to each part.

1 What is this thing, really?

Imagine a busy restaurant kitchen during a dinner rush. Orders come in. Cooks pick them up and make the food. Some cooks are fast, some are slow. Sometimes a cook suddenly quits and walks out in the middle of cooking a dish. When it gets very busy, the manager pulls in extra cooks; when it quiets down, the manager sends some cooks home. And there is a big screen on the wall showing you, in real time, every order and every cook.

That is exactly what this project is - except instead of a kitchen making food, it is a **computer system making pictures**. And instead of you having to imagine it, the screen is real: you open a web page and watch the whole thing happen, over and over, forever.

THE KITCHEN

Throughout this document, the **kitchen** is our picture of the system. Orders are jobs to do. Cooks are little worker programs. The manager is the part that keeps the whole thing running. We will use this same kitchen the whole way through, so it is worth holding onto.

The project is called a **render farm**. "Render" is the word for a computer turning instructions into a finished picture (the way a movie studio "renders" each frame of an animated film). A "farm" just means a lot of computers doing that work together. So a render farm is a crowd of computers cooperating to turn out picture after picture.

One important, honest detail: in this demo the cooks do not really cook. They *pretend* to work for a few seconds, the way an actor pretends to chop vegetables. Everything *around* the pretending is genuinely real, though - the orders are real, the cooks are real separate programs, and when a cook "quits," it really does stop existing. We will come back to why that distinction matters. For now: the work is pretend, but the teamwork is real.

2 A few words we will need

Before we go further, here are the plain-language meanings of the words that will keep coming up. You do not need to memorize these; just glance at them, and come back if a word later feels slippery.

Program

A set of instructions a computer follows. A recipe, basically. On its own it is just words on paper.

Process

A program that is *actually running right now*, doing its thing. The difference between a program and a process is the difference between a recipe in a book and a cook actively following that recipe at the stove. In our kitchen, **each cook is a process**. They are genuinely separate - one cook quitting does not stop the others, exactly like one running program crashing does not crash the others.

Server

A program (a process) whose whole job is to wait for requests and answer them. "Serve" as in a waiter serving tables. The manager of our kitchen is a server.

Queue

A line of things waiting their turn, like people lined up at a coffee shop. New items join the back; workers take items from the front. In the kitchen this is the **rail of order tickets** waiting to be cooked.

Job (or "frame")

One single piece of work to be done. In this system one job is one **frame** - one picture to produce. Think of it as one order ticket.

Worker

A process whose job is to take work off the queue and do it. Our **cooks** are workers.

Redis

A shared, super-fast scratchpad that every program can read from and write to at once. Picture a big whiteboard bolted to the kitchen wall that every cook and the manager can all see and update. Redis is where the order rail actually lives, so that all the separate cooks are looking at the *same* list of orders.

WebSocket

A phone line between two programs that stays open, so either side can talk the instant it has something to say, without hanging up and redialing. This is the always-on connection between the kitchen and the screen on the wall.

Snapshot

A photograph of the entire situation at one moment - every order, every cook, everything - frozen in time. The screen is really just a rapid slideshow of these snapshots.

WHY "SEPARATE" KEEPS GETTING EMPHASIZED

A lot of this document leans on the idea that the cooks are *truly separate* running programs, not pretend ones drawn on a screen. That separateness is the whole point of the project. It is what makes a crash a real crash and recovery a real recovery, instead of a magic trick. Keep an eye out for the word "separate" - it is doing a lot of work.

3 Who is in the kitchen

Our system has four kinds of characters. Here is each one, what it does, and the real name it has in the code.

The whiteboard on the wall - Redis

Everything the kitchen shares lives on one big whiteboard that everyone can see: the list of orders waiting, and little status notes the cooks scribble about what they are doing. The cooks never talk to each other directly. They only read and write the whiteboard. This is powerful: because there is one shared whiteboard, any cook can pick up any order, and nobody needs to be told who is doing what.

THE REAL THING

The whiteboard is **Redis**. The order rail is a queue that runs on top of Redis, managed by a helper called **BullMQ** (think of BullMQ as the set of rules for how tickets get hung up, grabbed, and checked off). The scribbled status notes travel on a separate part of the whiteboard called a "channel."

The manager - the orchestrator

One character watches over everything. The manager does not cook. Instead, the manager: puts fresh orders onto the rail, hires new cooks when orders pile up, sends cooks home when it is quiet, occasionally causes a cook to quit on purpose (to show off how the kitchen recovers), keeps a running story of everything that happens, and feeds the screen on the wall.

THE REAL THING

The manager is the **orchestrator** - a server program. Inside it lives a smaller brain called the **Director** that makes the moment-to-moment decisions. "Orchestrator" is a good word for it: like an orchestra conductor, it does not play an instrument, it coordinates everyone who does.

The cooks - workers

The cooks do the actual work. Each cook grabs one order at a time from the rail, makes it in two steps (more on that soon), scribbles progress on the whiteboard as they go, and then grabs the next order. Each cook is a completely separate person - if one faints, the others keep cooking.

THE REAL THING

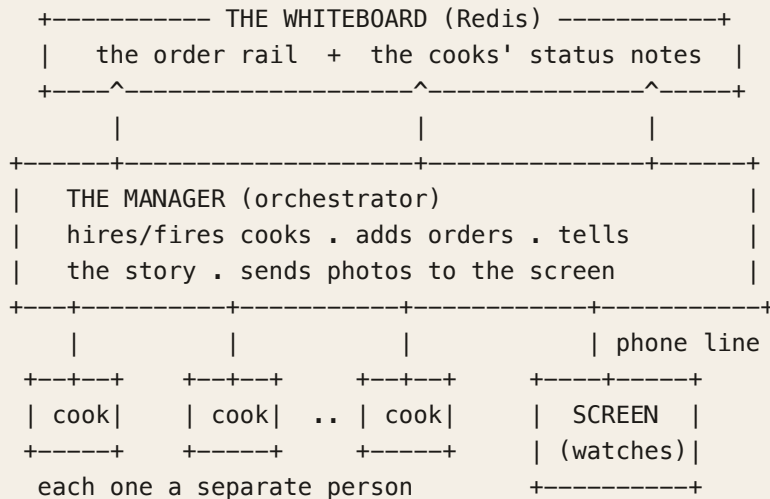
Each cook is a **worker process**. The manager creates them by "forking," which is the computer word for starting a brand-new separate program. Each cook has their own private connection to the whiteboard.

The screen on the wall - the web page

Finally there is the big screen showing the whole kitchen live. The screen only *watches*. It cannot cook, cannot touch orders, cannot change anything. It just shows you what is happening and gives you a few buttons to poke the kitchen with.

THE REAL THING

The screen is a **web page** (a "single-page app") in your browser. It receives snapshots over the WebSocket phone line and draws them. It never touches Redis and holds no secrets - closing it changes nothing about the running kitchen.



4 The journey of one order

Let us follow a single order from start to finish, slowly. This is the heartbeat of the whole system, so it is worth taking our time.

Step one - the order appears. The manager writes a new order ticket and hangs it on the rail. Right now it is just waiting; no cook has touched it. On the screen this shows up as a card in the first column, labeled **QUEUED** (which simply means "waiting in line").

Step two - a cook grabs it. The moment any cook is free, they take the next ticket off the rail. Nobody assigned it to them; they just took the next one available. That order's card now moves out of the waiting column.

Step three - the work, in two halves. Making the picture happens in two steps, one after the other:

- **RENDERING** - the first half. In our kitchen, think of this as "cooking the main part of the dish." It takes a couple of seconds. As the cook works, they keep scribbling a percentage on the whiteboard: 10% done, 40% done, 80% done. The screen shows this as a little filling progress bar.
- **COMPOSITING** - the second half. Think of this as "plating and garnishing" - assembling the finished result. It also takes a couple of seconds and also fills up a progress bar.

Step four - done. When both halves are finished, the order is complete. Its card slides into the final column, **DONE**, and the cook is immediately free to grab the next ticket.

waiting in line		first half		second half		finished
[QUEUED]	->	[RENDERING]	->	[COMPOSITING]	->	[DONE]
		~2.5 seconds		~2.5 seconds		
		progress bar		progress bar		
		fills 0..100%		fills 0..100%		

WHY TWO HALVES INSTEAD OF ONE

Splitting the work into two visible steps is a deliberate choice for the demo: it means there is always motion on the screen. An order is either waiting, in its first half, in its second half, or done - and it is constantly sliding from one to the next, so the picture never sits still.

THE REAL THING

The four columns - QUEUED, RENDERING, COMPOSITING, DONE - are the real "stages" a frame passes through, and they are the four columns of the board you see. Each half is roughly 2.5 seconds of pretend work, so a whole frame takes about 5 seconds from start to finish.

5 How the work spreads out by itself

Here is something that seems like it should need a boss barking assignments, but does not. Nobody ever tells a cook *which* order to make. There is just the one shared rail, and the rule "when you are free, take the next ticket." That single rule is enough to spread the work evenly, all on its own.

Think about why. A fast cook finishes quickly, becomes free sooner, and so grabs more tickets. A slow cook holds their ticket longer and grabs fewer. Without anyone planning it, the work naturally flows to whoever has hands free. Busy cooks are left alone; idle cooks pull the load. The balance is not designed - it *emerges* from the shared rail.

THE REAL THING

The grown-up name for this is **load balancing**, and this is one of the four headline behaviors the demo shows off. The remarkable part is that there is no load-balancing code at all. There is only a shared queue and idle workers pulling from it. That is genuinely how many real distributed systems spread their work.

6 When a cook quits in the middle

This is the most important part, and the coolest, so we will go very slowly.

Every so often - either because the manager decides to cause a little drama, or because you press the "Kill a node" button yourself - a cook is made to **quit instantly**. Not "finish up and leave." Not "hand off your dish." They vanish mid-cooking, holding a half-made order, with no goodbye and no cleanup. Their card on the screen flashes red.

IN THE KITCHEN

Picture a cook who, right in the middle of a dish, drops everything and storms out the back door. The half-finished order is now sitting there, claimed by someone who is gone. If nothing were done, that order would be stuck forever - nobody else feels allowed to touch it, because as far as anyone knows, it "belongs" to the cook who left.

So how does the order get rescued? There is a simple, clever rule. When a cook takes an order, they put their name tag on it - a claim that says "I've got this." But the claim is not permanent; it is more like a sticky note that has to be *refreshed* every few seconds to stay valid. A cook who is actually working keeps refreshing it without thinking. A cook who has stormed out, of course, refreshes nothing.

So after a few seconds, the abandoned order's claim goes stale. Meanwhile, the other cooks are periodically glancing around for orders whose claims have gone stale. The moment one of them spots the orphaned order, they announce "this one was abandoned," put it back on the waiting rail, and it gets picked up fresh by whoever is next free. The picture finishes - just on a different cook.

```
0s   cook is working, refreshing its claim every moment
0s   * SIGKILL *   the cook vanishes, claim frozen mid-air
~4s  the stale claim finally expires (nobody refreshed it)
~4-6s another cook notices the orphan, puts it back in line
~6s+ a free cook grabs it and finishes it normally
      total rescue: about 4 to 8 seconds
```

THE REAL THING

Quitting instantly is a real **SIGKILL** - a computer signal that means "stop this program immediately, no chance to clean up." The name tag is a **lock**; refreshing it is renewing the lock; the claim going stale is the lock expiring after about 4 seconds. The other cooks glancing around is the **stalled-job check** that runs every 2 seconds. Add those up and a rescued order comes back within about 4 to 8 seconds. This whole recovery is done by the queue system (BullMQ) itself - it is not faked for the demo. The grown-up name for this behavior is **fault tolerance**: the system tolerates a part breaking without losing the work.

NOTHING EVER GETS TRULY LOST

An order can be rescued and re-tried many times if it keeps getting orphaned. There is a very generous limit before the system gives up. And even if an order somehow used up every retry, it still gets moved to the DONE column - just marked in red as "failed" - so the kitchen is never stuck waiting on an order that will never finish. The line always keeps moving.

7 Hiring and sending cooks home

A good kitchen manager watches how many orders are piling up and adjusts the number of cooks to match. Our manager does exactly this, using one simple number: **how many orders are currently waiting on the rail**.

- **Too many orders waiting?** If the pile of waiting orders grows past a certain size, the manager hires one more cook - starts up a brand-new worker to help clear the backlog. It will keep hiring, one cook at a time on later checks, up to a maximum of six cooks. More hands, faster clearing.
- **Hardly any orders waiting?** If the pile shrinks below a small threshold, the manager sends one cook home - but only a cook who is currently idle, never one in the middle of an order. It will keep trimming down to a minimum of two cooks, so there is always at least a small crew ready.
- **Somewhere in between?** The manager leaves the crew exactly as it is. It only changes the number of cooks by one at a time, and never hires and fires in the same breath, so the crew size cannot flip-flop wildly.

IN THE KITCHEN

Dinner rush hits, tickets stack up, the manager waves in extra cooks from the back. The rush passes, the rail empties out, the manager quietly tells the cooks who are standing around to clock out. The kitchen breathes in and out with the demand.

THE REAL THING

This is **autoscaling** - the system automatically growing and shrinking its own workforce based on load. Hiring a cook is "forking a new worker process"; sending one home is closing a worker gracefully. On the screen you literally watch cook cards appear when it is busy and disappear when it is calm, with the story log announcing each "autoscaling up" and "autoscaling down."

8 Rush orders that jump the line

Not every order is equal. Some are marked **rush** - VIP orders that should be made before the ordinary ones. When a rush order is hung on the rail, it does not go to the back of the line like

everyone else. It jumps toward the front, so the next free cook grabs it ahead of the ordinary orders that were already waiting.

One honest limit: a rush order cannot yank a dish out of a cook's hands. A cook who is already mid-order finishes that one. What a rush order jumps is the *waiting line*, not work already underway. So on screen you see a rush order slide up toward the front of the QUEUED column and start cooking sooner than its neighbors - but it politely lets in-progress dishes finish.

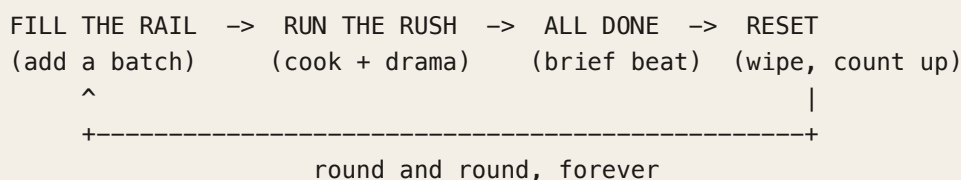
THE REAL THING

This is **job priority**, the fourth headline behavior. About one in seven orders is randomly marked rush. The queue itself understands priority and hands out the highest-priority waiting order first - so, just like load balancing, the line-jumping is a genuine property of the shared queue, not the screen rearranging cards to look pretty. Rush orders wear a distinct badge so you can spot them.

9 How it runs forever on its own

Nobody has to babysit this kitchen. It runs itself in a loop, over and over, forever. The manager follows a simple four-beat rhythm:

- **Beat 1 - fill the rail.** Starting from an empty kitchen, the manager hangs up a fresh batch of orders (about sixteen), a few of them marked rush.
- **Beat 2 - run the rush.** The cooks work through the orders. Every five to ten seconds, the manager stirs in a bit of drama: maybe it hires a cook, maybe it sends one home, maybe it makes a cook quit so you can watch the rescue. This is the beat you spend the most time watching.
- **Beat 3 - all done.** When every order has reached the DONE column and the rail is empty, the kitchen pauses for a satisfied moment.
- **Beat 4 - reset.** The manager wipes the board clean, counts up one (cycle 1, cycle 2, cycle 3...), and goes right back to Beat 1.



YOU CAN GRAB THE WHEEL TOO

The four buttons on the screen - Pause, Inject more orders, Kill a cook, and Reset - poke the very same machinery the manager uses on its own. Pausing freezes only the manager's *automatic* drama; the cooks keep finishing what they were doing, and you can still personally cause a cook to quit even while paused. Your hand and the manager's hand pull the exact same levers.

THE REAL THING

The four-beat rhythm is called a **cycle**, and the little brain running it is the **Director**. A neat design detail: all of the Director's *decisions* are made by a small, self-contained piece of logic that just reads the situation and says what should happen, while a separate piece actually does it. Keeping "deciding" and "doing" apart is what makes the manager's brain easy to test and trust.

10 How we get to watch it all

How does the picture on the screen stay so up to date? The manager takes a full photograph of the whole kitchen - every order, every cook, every progress bar, the running story - about five times every second, and sends each photo down the open phone line to the screen. The screen just shows you the newest photo. Five photos a second is fast enough that it looks like smooth, live motion.

Sending the *whole* photo each time, rather than only "what changed," sounds wasteful, but at this small size (a couple dozen orders, a handful of cooks) a photo is tiny. And it comes with a lovely bonus: if you open the screen halfway through a rush, the very first photo you receive already shows the complete, correct picture. There is no awkward moment of catching up.

IN THE KITCHEN

Think of a security camera pointed at the kitchen, feeding a live picture to a monitor. The monitor does not need to understand cooking; it just faithfully shows whatever the camera sees, moment to moment. Walk up to the monitor at any time and you instantly see the true current state.

There are two more little pieces worth naming, because they are where the picture actually comes from:

- **The order rail tells the manager where each order is** - waiting, being worked on, finished, or abandoned. This is the trustworthy backbone of the picture.
- **Each cook shouts out what they personally are doing** - "I'm node three, I'm on order 12, I'm 60% through the second half" - by scribbling on the whiteboard. This is how the manager

knows the fine details each photo needs, like which cook owns which order and how full each progress bar is.

THE REAL THING

The photo is a **WorldState snapshot**; five-per-second is the broadcast rate; the open phone line is a **WebSocket**. The two sources - the queue's order lifecycle and the cooks' shouted status - are the system's "two channels of truth," which the manager blends into each single photo. And if the phone line drops, the screen quietly keeps redialing until it reconnects, so a hiccup heals itself.

11 Why go to all this trouble?

A fair question: if the cooking is pretend anyway, why bother with truly separate cooks, a real shared whiteboard, and real quitting? Why not just draw a cartoon of a kitchen that only *looks* like this is happening?

Because a cartoon would be a lie, and this is meant to teach the truth. The interesting behaviors here - work spreading itself out, a crash being survived, the crew growing and shrinking, rush orders cutting the line - are not things someone drew. They *happen*, as real consequences of real separate programs sharing one real queue. When you watch a rescued order come back, you are watching an actual recovery mechanism do its actual job, not an animation of one.

That is the entire spirit of the project: make something you can *watch* and *poke*, where the only thing faked is the boring part (the cooking), and everything that makes it a genuine distributed system is honestly real. The value is not the pictures it "renders" - it is seeing, with your own eyes, how a crowd of independent programs can cooperate, stumble, recover, and keep going.

THE ONE-SENTENCE VERSION

It is a real, tiny distributed system dressed up as a render farm, running itself in a loop so you can watch load balancing, crash recovery, autoscaling, and priorities actually happen - live, on a screen, forever.

12 The whole thing, in one breath

What is it?

A pretend render farm that behaves like a real distributed system, so you can watch a crowd of programs cooperate.

Who is in it?

A shared **whiteboard** (Redis) holding the order rail; a **manager** (the orchestrator, with its Director brain); several **cooks** (worker processes); and a **screen** (the web page) that only watches.

What happens to one order?

It appears on the rail (QUEUED), a free cook grabs it, works it through two halves (RENDERING then COMPOSITING), and it lands in DONE - about five seconds, start to finish.

What are the four things it shows off?

- **Work spreading by itself** (load balancing) - one shared rail; free cooks pull the next order, no boss needed.
- **Surviving a crash** (fault tolerance) - a cook quits; its abandoned order's claim goes stale and another cook rescues it.
- **Growing and shrinking the crew** (autoscaling) - hire when orders pile up, send idle cooks home when it is calm.
- **Rush orders jumping the line** (priorities) - VIP orders cut ahead of waiting ones, but let in-progress dishes finish.

How do we see it?

The manager photographs the whole kitchen five times a second and sends each photo down an always-open phone line to the screen, which shows the newest one.

And then?

The rail empties, the manager wipes the board, counts up one, refills it, and the whole thing goes again. Forever.